MobaSSL: An Adaptive Graph Self-Supervised Learning Framework Emphasizing Functional Substructure

Wenhao Yu
Beihang University
Beijing, China
yuwenhao@buaa.edu.cn

ABSTRACT

In recent years Graph Neural Networks (GNNs)[11, 28, 36] are increasingly being employed to process structural data prevalent in diverse domains such as social networks, molecular graphs, and multi-agent networks. In order to effectively extract valuable insights concealed within the structures, some researchers meticulously integrate experience-based rules into their encoders. However, these approaches often lead to a lack of generality and flexibility in rule selection.

This paper introduces a comprehensive GNNs-based Encoding Framework, designed to address these limitations by focusing on functional substructures within ubiquitous graph architectures. Specifically, we adopt the widely used fragmentation rule, BRICS, as our foundational approach to identifying functional motifs within molecular graphs. We propose a neural networks (NNs) based controller that enables node-wise adaptation of BRICS during the message-passing process of GNNs. The controller is further divided into two components: switchers and a decider, each fulfilling specific roles in transforming the meaning of each embedding layer and determining the subsequent layer. Extensive experiments conducted on various downstream benchmark tasks demonstrate that our framework consistently outperforms state-of-the-art baselines.

CCS CONCEPTS

• **Computing methodologies** → **Artificial intelligence**; • **Mathematics of computing** → *Graph algorithms*; • **Information systems** → *Information retrieval*.

KEYWORDS

Graph Neural Networks, Network Motif, Neural Architecture Search

ACM Reference Format:
Wenhao Yu. 2023. MobaSSL: An Adaptive Graph Self-Supervised Learning Framework Emphasizing Functional Substructure. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (Conference acronym 'XX).* ACM, New York, NY, USA, 7 pages. https://doi.org/XXXXXXX.XXXXXXX

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2023 Association for Computing Machinery.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

Unpublished working draft. Not for distribution.

1 INTRODUCTION

Graph Neural Networks (GNNs) have emerged as the predominant approach for representation learning of structural data. Most GNNs involve message passing and information aggregation from local neighborhoods, enabling the framework to capture higher-order information embedded in the entire graph.

Graphs are pervasive in the real world and manifest in diverse contexts such as biochemical molecules, social networks, citation networks, and multi-agent networks [3, 15, 34, 38]. Various tasks built upon these datasets, such as node classification, graph classification, link prediction, and graph generation[2, 8, 11, 41], pose challenges due to the intricate and diverse graph topologies. Despite GNNs consistently outperforming unstructured neural network designs, they grapple with issues of over-smoothing and over-fitting when attempting to encapsulate both local and global information within realistic graphs[42]. Human-designed rules[1, 43] have been incorporated to guide GNNs in recognizing functional substructures during the learning process. However, these approaches are confined to their specifically designed neural architectures and rigid rule establishment, limiting their effectiveness when attempting to capture complex structures in a broader dataset.

On another front, Graph Neural Architecture Search (GraphNAS) methods[4, 16, 17, 35, 44, 46] have garnered significant attention due to their adaptability and versatility in representation learning on graphs. These methods focus on devising a well-defined action list encompassing the entire architectural space, comprising permutations of all component options. However, they heavily rely on the meticulous design of the option list[4], often borrowing from NAS methods in Computer Vision and Natural Language Processing. Moreover, these methods overlook the necessity of mining higher-order information with shallow Graph Neural Networks.

In this paper, we present **MobaSSL**, a **Mo**tif-**b**ased **A**daptive **S**elf-**S**upervised **L**earning framework designed to jointly learn effective GNNs and motif-view graphs associated with the GNNs. The encoder of MobaSSL consists of GNNs and a node-wise controller, aiming to guide these architectural components collaboratively and enhance the quality of both embeddings and motif-view graphs. MobaSSL first undergoes pretraining on a larger dataset to acquire transferable GNNs and a basic controller. Subsequently, a pre-finetuning process prunes the GNNs and the basic controller further, based on the specific downstream dataset and task. Finally, the controller is frozen and generates stable motif-view graphs for the downstream dataset, enabling the GNNs to learn from functional substructures in the finetuning phase. Our contributions are summarized as follows:

- We explore the possibility of integrating complex rules for selecting functional substructures into vanilla GNNs by appending a controller to the GNNs.
- We employ the policy gradient method to jointly update the parameters of GNNs and the controller. This workflow is generalizable, holding the potential to incorporate more intricate selection rules into the entire framework.
- We validate the performance of MobaSSL on various chemical benchmark tasks. The results demonstrate that MobaSSL can identify an effective graph neural architecture that rivals human-designed architecture in terms of accuracy on test sets.

2 PRELIMINARIES

2.1 Molecular Property Prediction

Molecular Property Prediction (MPP) is a critical research area in domains such as chemistry, physics, and materials science [31]. The objective in MPP is to utilize data-driven approaches to predict diverse properties of molecules, including binding affinity with proteins, toxicity, solubility, and more. MPP finds wide-ranging applications in drug discovery, quantum chemistry, and biology [10, 13, 18]. Many studies treat molecules as graphs to comprehensively incorporate internal spatial and distance information of atoms in molecules [5, 23, 37], where nodes correspond to atoms and edges correspond to bonds. GNNs have gained significant traction in MPP, effectively capturing the structural and chemical information of molecules [5]. To leverage rings in compounds more effectively, an approach proposed by [47] thoughtfully incorporates ring representation into their architecture. Furthermore, graph-based transformers [21] are introduced to achieve self-supervised learning on large-scale molecular data.

In order to uncover functional substructures within molecule graphs, [43] employs an enhanced version of the Breaking of Retrosynthetically Interesting Chemical Substructures (BRICS) algorithm to decompose molecular graphs into meaningful motif trees and reconstruct these trees in the pre-training stage. Additionally, [25] views motifs as subgraphs and guides the encoder to learn meaningful motif representations during the pre-training stage.

2.2 Graph Neural Networks and Architecture Search

2.2.1 Graph Neural Networks. Graph Neural Networks (GNNs) are a class of neural networks that operate on graph-structured data, such as social networks, citation networks, and knowledge graphs. A graph, denoted as $G = (\mathcal{V}, \mathcal{E})$, consists of a set of nodes $\mathcal{V}$ and a set of edges $\mathcal{E}$ representing the relationships between the nodes. GNNs typically consist of multiple layers, each of which updates the node representations by aggregating information from their neighbors. The aggregation function can be different for each layer and is chosen from a predefined set of operators, such as graph convolutional networks (GCN) [11], graph attention networks (GAT) [28], or graph isomorphism networks (GIN) [36]. The choice of the aggregation function, as well as other hyperparameters such as the number of layers (L), the number of hidden units, and the activation function, can significantly affect the performance of GNNs on different tasks and datasets.

Message passing is a fundamental concept in GNNs, where information is propagated through the edges of the graph to update node representations. In each layer (l) of a GNN, the message passing can be defined as follows:

$$\begin{aligned}
\hat{\mathbf{h}}_v^{(l+1)} &= \textsc{Aggregate}\left(\left\{\mathbf{h}_u^{(l)}, \forall u \in \mathcal{N}(v)\right\}\right); \\
\mathbf{h}_v^{(l+1)} &= \textsc{Update}\left(\mathbf{h}_v^{(l)}, \hat{\mathbf{h}}_v^{(l+1)}\right); \\
\mathbf{h}_{\mathrm{G}} &= \textsc{Readout}\left(\left\{\mathbf{h}_v^{(L)}, \forall v \in \mathcal{V}\right\}\right),
\end{aligned}$$

where $\mathcal{N}(v)$ is the set of neighbors of node v; L is the total number of layers in the GNN; $\mathbf{h}_v^{(l)}$ represents the representation of node v in layer l; $\mathbf{h}_v^{(L)}$ are the node representations from the final layer. $\hat{\mathbf{h}}_v^{(l+1)}$ is the temporary aggregated information for node v at layer $l+1$; $\mathbf{h}_{\mathrm{G}}$ is the graph-level representation; Aggregate is the aggregation function that aggregates information from neighboring nodes using $\mathbf{h}_u^{(l)}$ for u in $\mathcal{N}(v)$; Update is a function that combines the aggregated information ($\hat{\mathbf{h}}_v^{(l+1)}$) with the previous representation of the node ($\mathbf{h}_v^{(l)}$), followed by a linear transformation like Multilayer Perceptron(MLP); Readout is the readout process generating a representation that encapsulates the entire graph's information, which is used for making predictions for downstream tasks.

2.2.2 Graph Neural Architecture Search. GraphNAS aims to find the best GNN architecture for a given task and dataset by exploring a large and discrete search space of possible architectures. GraphNAS can be formulated as an optimization problem:

$$\max_{\alpha} R(\alpha; D_{val}) \quad \text{s.t.} \quad \alpha \in \mathcal{A},$$

where α is an architecture in the search space $\mathcal{A}$, $R(\alpha; D_{val})$ is the reward function that measures the performance of α on a validation dataset D_{val}, and $\mathcal{A}$ is defined by a set of rules that specify the valid architectures. For example, $\mathcal{A}$ can be defined by a grammar that generates variable-length strings that describe the architectures of GNNs [4]. Alternatively, $\mathcal{A}$ can be defined by a directed acyclic graph (DAG) that represents the topology of GNNs [44].

GraphNAS is a challenging problem because the search space $\mathcal{A}$ is usually very large and discrete, which makes it difficult to apply gradient-based optimization methods. Moreover, evaluating the reward function $R(\alpha; D_{val})$ requires training and testing each candidate architecture α on a given graph dataset, which can be very time-consuming and computationally expensive. Therefore, GraphNAS methods often employ various techniques to reduce the search cost and improve the search efficiency, such as reinforcement learning [4], evolutionary algorithms [46], progressive pruning [16], Monte-Carlo tree search [17], or linearization [35].

2.3 Graph Self-supervised Learning

Graph Self-supervised Learning (GSSL) is a subset of graph representation learning that operates on unlabeled graph data, aiming to unveil inherent graph characteristics without relying on external supervision. This branch of learning is vital for a multitude of applications that necessitate effective graph representations, including node classification, graph classification, and link prediction [33].

GSSL trains models in a self-supervised manner by predicting missing or corrupted attributes, structures, or contexts within the

Unpublished working draft. Not for distribution.

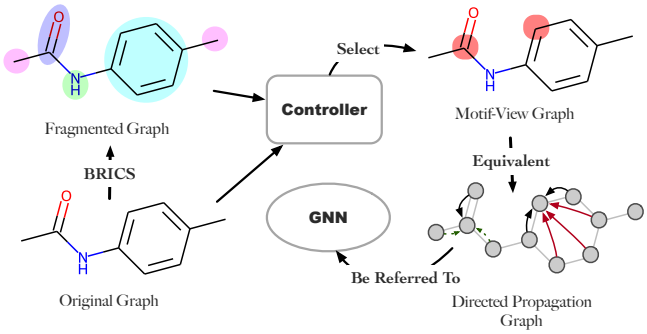

Figure 1: Derivation of Motif-View Graph and Directed Propagation Graph (DPG). The atoms highlighted by red circles in the motif-view graph indicate nodes that will aggregate information from the located motif. The red solid line and green dashed line in the DPG represent additional and excluded information, respectively, compared to the original propagation of G_o.

graph. These tasks compel the model to capture pertinent graph information transferable to downstream applications. GCC (Graph Contrastive Coding) [19] leverages random walks to generate positive and negative views, training a GNN encoder to distinguish them. Graphite [6] utilizes a variational autoencoder to reconstruct subgraphs sampled from the graph, regularizing the latent space with adversarial training. InfoGraph [26] predicts mutual information between node-level and graph-level representations using a bilinear discriminator.

3 METHODOLOGY

In this section, we will introduce MobaSSL at three different levels, ranging from the specific to the comprehensive. In § 3.1, we will demonstrate how to construct a motif-view graph using the original graph, BRICS rule, and our controller. In § 3.2, we will provide a detailed introduction to the design of the controller and explain how to update it in various settings. Finally, in § 3.3, we take a higher-level perspective to explore the MobaSSL framework, emphasizing the incorporation of pre-finetuning and frozen graph techniques to enhance overall design stability.

3.1 Adaptive Motif-View Graph

At a granular level, we introduce the concept of the *Motif-View Graph*, an adaptive and trainable perspective of the original graph. This representation aims to accentuate functional substructures within a specific graph of a particular dataset.

To effectively identify these functional substructures, an efficient human-designed rule is initially introduced to segment the entire graph into distinct subgraphs. For instance, in the case of a molecular graph, we employ a fragmentation algorithm called BRICS [1] and a post-processing procedure [43] on BRICS. These steps decompose the graph into various motifs (i.e., subgraphs). The enhanced BRICS algorithm has demonstrated its efficacy in reducing the motif vocabulary's size and enhancing motif occurrence frequency, resulting in a motif tree of meaningful size for a given graph. This meaningful fragmentation ensures generalization from the upstream dataset to the downstream [43].

Fig. 1 shows the process of derivation of various views of graphs. The Original Graph is initially fragmented into various motifs. The motif-view graph is then formed based on the selection made by the controller. The atoms highlighted by red circles in the motif-view graph indicate nodes that will aggregate information from the located motif. Finally, the generated directed propagation graph is passed to the GNN for the message-passing process.

For the message passing process, we define an action $a_v^{(l)} \in \{0, 1\}$ for node v in the l-th layer. Here, $a_v^{(l)} = 0$ implies that node v aggregates information from its immediate neighbors under the controller's guidance, while $a_v^{(l)} = 1$ denotes aggregation from all nodes within the same motif as node v. With actions for all nodes in the l-th layer denoted by $\mathbf{a}^{(l)}$, a directed propagation graph (DPG) $G_{\mathbf{a}^{(l)}}$ is formed.

Defining a set $\{\mathbf{a}^{(l)}\}$ based on permutations of actions in the l-th layer, we obtain a corresponding DPG set $\{G_{\mathbf{a}^{(l)}}\}$. We refer to the sequential Cartesian product of $\{\mathbf{a}^{(l)}\}$ as the action space. For a given original graph G_o, we formulate the action space $\boldsymbol{a}$ as:

$$\boldsymbol{a} = \Pi_{l=1}^{L}\{\mathbf{a}^{(l)}\} = \{\mathbf{a}^{(1)}\} \times \{\mathbf{a}^{(2)}\} \times \cdots \times \{\mathbf{a}^{(L)}\}, \tag{1}$$

Similarly, we define the DPG space $\boldsymbol{G}$ as:

$$\boldsymbol{G} = \Pi_{l=1}^{L}\{G_{\mathbf{a}^{(l)}}\} = \{G_{\mathbf{a}^{(1)}}\} \times \{G_{\mathbf{a}^{(2)}}\} \times \cdots \times \{G_{\mathbf{a}^{(L)}}\}. \tag{2}$$

As per the given definition, the cardinalities of both spaces amount to $2^{N\times L}$, where N represents the number of nodes and L signifies the number of layers.

Finally, for a given controller that makes specific decisions for each node in each layer, a sequence of action vectors for each layer is determined as $(\mathbf{a}^{(l)})_{l=1}^{L} = (\mathbf{a}^{(1)}, \mathbf{a}^{(2)}, \cdots, \mathbf{a}^{(L)})$. Consequently, the sequence of directed propagation graphs $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$ is also defined by the controller. $(\mathbf{a}^{(l)})_{l=1}^{L}$ and $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$ represent a point in $\boldsymbol{a}$ and $\boldsymbol{G}$, respectively.

In conventional GNNs, the sequence of DPGs used to aggregate information duplicates the original graphs $(G_o)_{l=1}^{L}$ (where the undirected original graph can be viewed as a bi-directed graph). This implies that the information is always obtained from 1-hop neighbors for a single aggregation. Given a certain experimental setting, we assume that the performance on the target downstream task is solely influenced by the DPG sequence. Thus, we define an injective function $\mathcal{P} : \boldsymbol{G} \to [0, 1]$ on the DPG space $\boldsymbol{G}$. We aim to present two evident theorems:

Theorem 3.1. *There exists* $(G^*_{\mathbf{a}^{(l)}})_{l=1}^{L} \in \boldsymbol{G}$ *such that*

$$\mathcal{P}((G^*_{\mathbf{a}^{(l)}})_{l=1}^{L}) \geq \mathcal{P}((G_o)_{l=1}^{L}). \tag{3}$$

Theorem 3.2. *There exists* $(\overline{G}_{\mathbf{a}^{(l)}})_{l=1}^{L}$ *such that, for any* $(G_{\mathbf{a}^{(l)}})_{l=1}^{L} \in \boldsymbol{G}$,

$$\mathcal{P}((G_{\mathbf{a}^{(l)}})_{l=1}^{L}) \leq \mathcal{P}((\overline{G}_{\mathbf{a}^{(l)}})_{l=1}^{L}) \leq 1. \tag{4}$$

Proof. Theorem 3.1 holds because $(G_o)_{l=1}^{L}$ is a point in the space $\boldsymbol{G}$. Theorem 3.2 holds because $\mathcal{P}$ is injective, and the cardinality of the range in $\boldsymbol{G}$ is no larger than $2^{N\times L}$. Hence, the upper bound ought to be reached, namely, $\mathcal{P}((\overline{G}_{\mathbf{a}^{(l)}})_{l=1}^{L})$. □

Unpublished working draft. Not for distribution.

With the proof provided above, two key questions emerge for us to address:

- How can we design a learnable controller that effectively covers the entire Directed Propagation Graphs (DPGs) space $\boldsymbol{G}$?
- How can we efficiently update the controller in both a general setting and a specific setting?

These two questions are answered in § 3.2 and § 3.3.

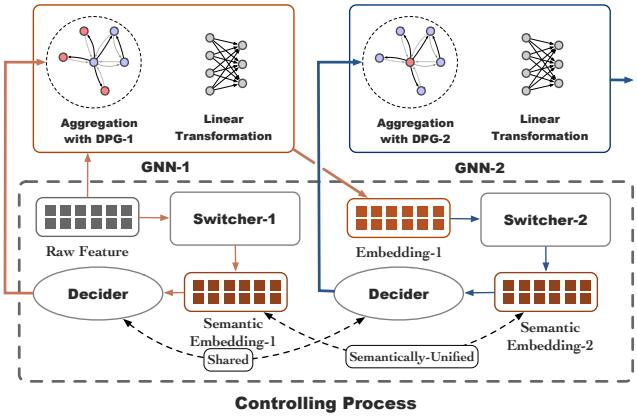

Figure 2: Workflow of a 2-layer encoder, illustrating the design of the Controller and Graph Neural Networks (GNNs). The upper half demonstrates that GNNs aggregate information according to the sampled Directed Propagation Graph (DPG) instead of the original graph G_o, while the lower half clarifies the controlling process with embeddings as input.

3.2 Design and Update of Controller

3.2.1 Design of Controller. To achieve comprehensive coverage of the entire Directed Propagation Graphs (DPGs) space $\boldsymbol{G}$, we employ a Neural Networks-based Controller. Given the magnitude of the space, the design of the controller must meet two crucial criteria. Firstly, the input space must possess sufficient distinctiveness to enable the controller to differentiate between various nodes within a layer, as well as a single node across different layers. Secondly, the controller should recognize the significant disparities among nodes and among layers, ensuring a judicious search outcome within the constraints of time and parameter limitations.

To meet the first criterion, we utilize the embeddings output by GNNs as the input to the controller. GNNs like Graph Isomorphism Networks (GINs) have demonstrated higher expressiveness in distinguishing specific graph structures compared to some kernel-based methods [36]. Moreover, studies [40, 45] have revealed that the layers in deep neural networks have distinct responsibilities, resulting in the distinctiveness of the output embeddings.

The design of the controller is inspired by the second criterion and the input format of embeddings. As depicted in Fig. 2, the controller primarily consists of L Switchers and a Decider. The number of Switchers matches the layers in GNNs, each of which transforms the embedding output from the corresponding GNN layer to a unified semantic level. Following that, the Decider makes decisions for all nodes in the specific layer. Finally, GNNs refer to the DPGs $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$ generated from the sampled actions $(\mathbf{a}^{(l)})_{l=1}^{L}$ by the Decider. We formulate the entire decision-making process as follows:

$$\begin{aligned}
\mathbf{H}^{(0)} &= \mathbf{X}, \\
\mathbf{S}^{(l)} &= \textsc{Switcher}^{(l)}(\mathbf{H}^{(l-1)}), && (5)\\
G_{\mathbf{a}^{(l)}} \sim \mathbf{a}^{(l)} &= \text{sample}(\textsc{Decider}(\mathbf{S}^{(l)})), && (6)\\
\mathbf{H}^{(l)} &= \text{GNN}^{(l)}(\mathbf{H}^{(l-1)}, G_{\mathbf{a}^{(l)}}), && (7)\\
\mathbf{h}_{\mathrm{G}} &= \textsc{Readout}\left(\mathbf{H}^{(L)}\right),
\end{aligned}$$

where $\mathbf{X}$ is the raw feature matrix of nodes, $\mathbf{H}^{(l)}$ represents the embeddings of nodes in the l-th layer, $\mathbf{S}^{(l)}$ is the semantically unified matrix derived from $\mathbf{H}^{(l-1)}$, $\mathbf{a}^{(l)}$ stands for the action vector sampled from the output logits of $\textsc{Decider}(\mathbf{S}^{(l)})$, $G_{\mathbf{a}^{(l)}}$ is the generated DPG corresponding to $\mathbf{a}^{(l)}$, which is employed in the aggregation process of the next layer $\text{GNN}^{(l)}$.

3.2.2 Update of Controller. The controller can be viewed as a policy net, which outputs the probability of different scopes, i.e., 1-hop neighbors or nodes in motif, of the specific node on the specific layer. The parameters of Controller and GNN can be named as θ and ϕ respectively. We formulate the output probabilities of the sampled actions in the l-th as $p_{\theta}^{(l)}\left(a^{(l)}|\mathbf{H}_{\phi}^{(l-1)}\right)$. Then we can further transform $p_{\theta}^{(l)}\left(a^{(l)}|\mathbf{H}_{\phi}^{(l-1)}\right)$ into a more meaningful view:

$$\begin{aligned}
p_{\theta}^{(l)}\left(a^{(l)}|\mathbf{H}_{\phi}^{(l-1)}\right) &= p_{\theta,\phi}^{(l)}\left(a^{(l)}|a^{(l-1)}, \mathbf{H}_{\phi}^{(l-2)}, G_o\right) && (8)\\
&= p_{\theta,\phi}^{(l)}\left(a^{(l)}|a^{(l-1)}, a^{(l-2)}, \mathbf{H}_{\phi}^{(l-3)}, G_o\right) && (9)\\
&\cdots \\
&= p_{\theta,\phi}^{(l)}\left(a^{(l)}|(\mathbf{a}^{(j)})_{j=1}^{l-1}, \mathbf{H}^{(0)}, G_o\right) && (10)\\
&= p_{\theta,\phi}^{(l)}\left(G_{\mathbf{a}^{(l)}}|(G_{\mathbf{a}^{(j)}})_{j=1}^{l-1}, G_o, \mathbf{X}\right) && (11)\\
&= p_{\theta,\phi}^{(l)}\left(G_{\mathbf{a}^{(l)}}|(G_{\mathbf{a}^{(j)}})_{j=0}^{l-1}, \mathbf{X}\right). && (12)
\end{aligned}$$

where Eq. (8) to Eq. (10) are based on Eq. (7) and the generation of $G_{\mathbf{a}^{(l)}}$ from $\mathbf{a}^{(l)}$ in Eq. (6), Eq. (11) is derived from the bijective correspondence between $a^{(l)}$ and $G_{\mathbf{a}^{(l)}}$, Eq. (12) consider G_o as $G_{\mathbf{a}^{(0)}}$ and incorporate it into DPG sequence $(G_{\mathbf{a}^{(j)}})_{j=0}^{l-1}$.

Thus, the probability of the sampled DPG sequence $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$ can be calculated by the product of $p_{\theta}^{(l)}\left(a^{(l)}|\mathbf{H}_{\phi}^{(l-1)}\right)$:

$$\begin{aligned}
p_{\theta,\phi}\left((G_{\mathbf{a}^{(l)}})_{l=1}^{L}|\mathbf{X}, G_o\right) &= \Pi_{l=1}^{L} p_{\theta,\phi}^{(l)}\left(G_{\mathbf{a}^{(l)}}|(G_{\mathbf{a}^{(j)}})_{j=0}^{l-1}, \mathbf{X}\right). \\
&= \Pi_{l=1}^{L} p_{\theta}^{(l)}\left(a^{(l)}|\mathbf{H}_{\phi}^{(l-1)}\right).
\end{aligned}$$

We note $p_{\theta,\phi}\left((G_{\mathbf{a}^{(l)}})_{l=1}^{L}|\mathbf{X}, G_o\right)$ as $p_{\theta,\phi}\left((G_{\mathbf{a}^{(l)}})_{l=1}^{L}|G\right)$ for simplification, where we replace the raw feature $\mathbf{X}$ and the original graph structure G_o with G. Furthermore, we just use $p_{\theta,\phi}(G)$ to represent this sampling process of $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$ from G. Similarly, we name $R_{\phi}(G)$ as the whole reward obtained from GNN processing

Unpublished working draft. Not for distribution.

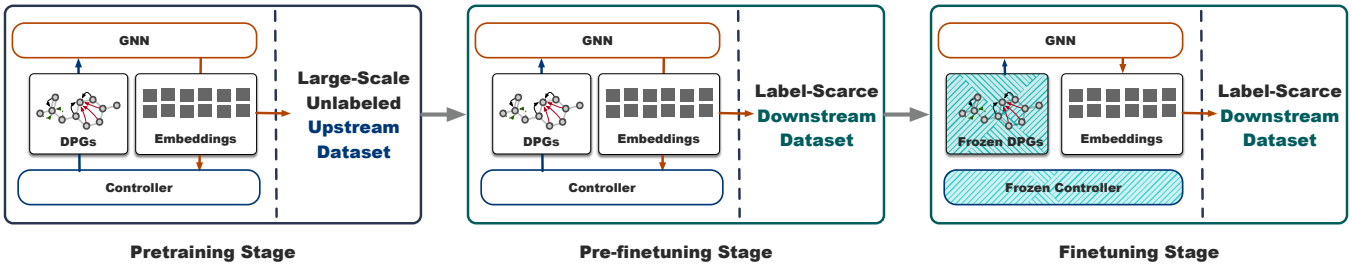

Figure 3: Overall Framework of MobaSSL

on G and the sampled $(G_{\mathbf{a}^{(l)}})_{l=1}^{L}$.

$$max_{\theta,\phi} \quad \mathbb{E}_G \mathbb{E}_{(G_{\mathbf{a}^{(l)}})_{l=1}^{L} \sim p_{\theta,\phi}(G)} (R_\phi(G)) \approx \Sigma_G R_\phi(G) \cdot p_{\theta,\phi}(G)$$

Here we use the training loss $L_\phi\ (G)$ as the negative reward, thus our goal is changed as:

$$min_{\theta,\phi} \quad \Sigma_G L_\phi\ (G) \cdot p_{\theta,\phi}(G)$$

The gradients on ϕ, θ are

$$\begin{aligned} &\nabla \Sigma_G L_\phi(G) \cdot p_{\theta,\phi}(G) \\ &= \Sigma_G (\nabla L_\phi(G) p_{\theta,\phi}(G) + L_\phi(G) \nabla (\log p_{\theta,\phi}(G)) p_{\theta,\phi}(G)) \\ &= \mathbb{E}_G (\nabla L_\phi(G) + L_\phi \nabla (\log p_{\theta,\phi}(G))) \\ &\approx \frac{1}{N} \Sigma_{n=1}^{N} (\nabla L_\phi(G_n) + L_\phi \nabla (\log p_{\theta,\phi}(G_n))), \end{aligned}$$

where N is the batch size, G_n is the n-th graph in the sampled batch from the whole dataset. Here we used the policy gradient method, which is effective in update the policy net in reinforcement learning[27]. In practical process, a baseline method is introduced to realize the stable update of controller. The gradient can be finally formulated as

$$\begin{aligned} &\nabla \Sigma_G L_\phi(G) \cdot p_{\theta,\phi}(G) \\ &\approx \frac{1}{N} \Sigma_{n=1}^{N} (\nabla L_\phi(G_n) + (L_\phi - \bar{L_\phi}) \nabla (\log p_\theta(G_n))), \end{aligned}$$

where $\bar{L_\phi}$ is the average loss in the batch.

3.3 Overall Framework of MobaSSL

The design of the controller requires obtaining the training loss to guide the updating process. Since the controller serves as a supplement to the GNNs encoder, our approach is not limited to specific tasks or training settings. For a comprehensive demonstration, we devised a self-supervised learning framework for graph-related tasks, which can be extended to other contexts.

Our framework primarily consists of three parts: pre-training, pre-finetuning, and fine-tuning. As illustrated in the Fig. 3, we start by jointly pretraining the GNNs and the controller on a larger upstream dataset. This pretraining process endows the controller with strong decision-making abilities and ensures that the output embeddings from GNNs are not only more effective in the pretext tasks but also better aligned with the controller. Moreover, in the pre-finetuning stage, the pretrained encoder is employed in downstream tasks, further enhancing its adaptation to specific settings. At this stage, a well-adapted controller is established. Subsequently, we fix the DPGs sequence $G^*_{\mathbf{a}^{(l)}}$ for each graph in the dataset based on the controller's output with the highest probability. Finally, during the fine-tuning stage, we disregard the controller and solely update the parameters of GNNs, which generate embeddings using the frozen $G^*_{\mathbf{a}^{(l)}}$. These embeddings are leveraged in subsequent downstream tasks.

4 EXPERIMENT SETUP

4.1 Datasets and Splitting

Our primary focus in this paper is on tasks related to molecular property prediction. In these tasks, a plethora of unlabeled molecules is available, while labeled data for downstream tasks is scarce. Specifically, for self-supervised pre-training tasks, we explore the whole framework in both transductive and inductive settings. In the inductive setting, we utilize 250k unlabeled molecules sampled from the ZINC15 database [24]. For the transductive setting, we use the combination of overall downstream training and validation graphs. For the downstream fine-tuning tasks, we consider 8 binary classification benchmark datasets from MoleculeNet [34]. As for the supervised learning settings, we examine the framework on these 8 datasets from MoleculeNet [34] without the pre-training stage.

To preprocess the SMILES strings from various datasets, we use the open-source package RDKit [12]. To simulate a real-world use case, we split the downstream dataset using scaffold-split [7, 20], a method that divides molecules based on their structures. We conduct three independent runs with random data splitting and report the means and standard deviations. The train/validation/test set split ratio is 80% : 10% : 10%.

4.2 Model Configuration

In our experiments, we employ a five-layer architecture based on Graph Isomorphism Networks (GINs) [36], a leading GNN method. Mean pooling is used as the Readout function for GIN. Same with GCL [39], we set the contrastive loss as the pretraining optimization goal. However, because not all the combinations of augmentations will outperform the no-augmentation GCL [39] for the specific dataset, we just select NodeDrop [39] and None-Augmentation as our final augmentation combination. Node features comprise atom numbers and chirality tags, while edge features encompass bond types and directions. During the pre-training phase, we conduct 100 epochs of GNN pre-training using the Adam optimizer with a learning rate of 0.001. In the pre-finetuning stage, we train for 50 epochs and select the best encoder based on the best validation

Unpublished working draft. Not for distribution.

epoch. For the fine-tuning phase, we train for 100 epochs and report the test scores based on the best validation performance. The hidden dimension for GINs is set to 300, and the dimension for the unified-semantic embedding output by Switchers is 150. The batch size is configured as 128 for pre-training and 32 for fine-tuning. All experiments are conducted using Tesla V100 GPUs.

4.3 Baselines

In this experiment, we conduct a comprehensive evaluation of MobaSSL against six state-of-the-art self-supervised methods specifically designed for Graph Neural Networks (GNNs). We categorize these methods into two contrast-based approaches (i.e., DGI, GCC[19, 29]), one prediction-based approach (i.e., GROVER[21]), and three generation-based approaches (i.e., AttrMasking, GPT-GNN, MGSSL[7, 8, 43]), according to the taxonomy proposed in [14]. Furthermore, we present the performance of MGSSL under two different generation orders, MGSSL-DFS and MGSSL-BFS.

Additionally, we include the GCL[39] baseline, using the same augmentations as MobaSSL. This comparison allows us to showcase the direct benefits of MobaSSL over conventional GCL.

5 DISCUSSION

Rule selections and Dataset Settings. In MobaSSL, we incorporate our controller into the conventional GCL self-supervised learning for graph classification downstream tasks. We aim to examine the validity of the adaptive motif-view graph which is referred to aggregate information from the same motif of the node. Actually, the concept of "recurring functional substructure" is not limited to "motif in the molecular graph". Some works[9, 32] delve into the complexities of the detection of network motifs, which are inspired by diverse organisms found from bacteria to humans. Work [22] enhances the convolutional neural networks on both homogeneous and heterogeneous citation nets with motifs integrated. Motifs can also be utilized to better judge the default from the relationship in trade nets[30]. Therefore, our proposed framework can be generalized to distinctive datasets where a reasonable rule detects recurring and functional patterns.

The difference between graph-level and node-level tasks. In MobaSSL, the downstream tasks consist exclusively of binary graph classification tasks. Nevertheless, the overall framework holds promise for enhancing node-level tasks owing to its node-distinguishing capability. We acknowledge that our methods could be further refined by considering the exclusion of readout operations, which might result in a trivial solution. We plan to investigate these aspects of our framework in future work.

6 CONCLUSION

In this study, we introduced MobaSSL, an innovative framework for self-supervised pre-training in Graph Neural Networks (GNNs). Our approach revolves around leveraging the inherent graph structure using Motif-View Graphs and Directed Propagation Graphs (DPGs). The Motif-View Graph accentuates functional substructures, while DPGs facilitate controlled information propagation within these substructures. We extensively investigated the design and impact of the controller in MobaSSL, emphasizing its composition of Switchers and a Decider, pivotal in generating DPGs for GNNs. This design has proven effective, showcasing the potency of using embeddings as input for the controlling process. Moreover, we thoroughly delineated the entire update process, affirming the controller's capability to find the best DPGs for different settings. Through a comprehensive exploration of various self-supervised pre-training methods, we demonstrated MobaSSL's superiority compared to state-of-the-art approaches. In future research, we aim to delve deeper into the effectiveness of the controller in diverse contexts. Additionally, we will strive to develop more efficient and effective update methods for our design.

REFERENCES

[1] Jörg Degen, Christof Wegscheid-Gerlach, Andrea Zaliani, and Matthias Rarey. 2008. On the Art of Compiling and Using'Drug-Like'Chemical Fragment Spaces. *ChemMedChem: Chemistry Enabling Drug Discovery* 3, 10 (2008), 1503–1507.

[2] Federico Errica, Marco Podda, Davide Bacciu, and Alessio Micheli. 2019. A fair comparison of graph neural networks for graph classification. *arXiv preprint arXiv:1912.09893* (2019).

[3] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *The world wide web conference*. 417–426.

[4] Hongyang Gao, Shuo Pei, Zhiting Huang, Zihang Dai, Ming-Fai Leung, Fanjin Chen, and Le Song. 2020. Graph neural architecture search. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 1403–1409.

[5] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International conference on machine learning*. PMLR, 1263–1272.

[6] Aditya Grover, Aaron Zweig, and Stefano Ermon. 2018. Graphite: Iterative Generative Modeling of Graphs. *ArXiv* abs/1803.10459 (2018).

[7] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. 2019. Strategies for pre-training graph neural networks. *arXiv preprint arXiv:1905.12265* (2019).

[8] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. 2020. Gpt-gnn: Generative pre-training of graph neural networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1857–1867.

[9] Nadav Kashtan, Shalev Itzkovitz, Ron Milo, and Uri Alon. 2004. Efficient sampling algorithm for estimating subgraph concentrations and detecting network motifs. *Bioinformatics* 20, 11 (2004), 1746–1758.

[10] Suyeon Kim, Dongha Lee, SeongKu Kang, Seonghyeon Lee, and Hwanjo Yu. 2023. Learning topology-specific experts for molecular property prediction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 8291–8299.

[11] Thomas N Kipf and Max Welling. 2017. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations (ICLR)*.

[12] Greg Landrum. 2013. *RDKit Documentation*. Release.

[13] Shengchao Liu, Meng Qu, Zuobai Zhang, Huiyu Cai, and Jian Tang. 2022. Structured multi-task learning for molecular property prediction. In *International conference on artificial intelligence and statistics*. PMLR, 8906–8920.

[14] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, and S Yu Philip. 2022. Graph self-supervised learning: A survey. *IEEE Transactions on Knowledge and Data Engineering* 35, 6 (2022), 5879–5900.

[15] Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. 2020. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 7211–7218.

[16] Zhenyu Liu, Xin Wang, Ziwei Zhang, Yijian Qin, and Wenwu Zhu. 2020. PSP: Progressive space pruning for efficient graph neural architecture search. In *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML-PKDD)*. 1–17.

[17] Zhenyu Liu, Xin Wang, Ziwei Zhang, Yijian Qin, and Wenwu Zhu. 2020. Search to aggregate neighborhood for graph neural network. In *International Conference on Data Mining (ICDM)*. 1–10.

[18] Chengqiang Lu, Qi Liu, Chao Wang, Zhenya Huang, Peize Lin, and Lixin He. 2019. Molecular property prediction: A multilevel quantum interactions modeling perspective. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 33. 1052–1060.

[19] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training. *ArXiv* abs/2010.13902 (2020).

[20] Bharath Ramsundar, Peter Eastman, Pat Walters, and Vijay Pande. 2019. *Deep learning for the life sciences: applying deep learning to genomics, microscopy, drug discovery, and more.* " O'Reilly Media, Inc.".

Unpublished working draft. Not for distribution.

[21] Yu Rong, Yatao Bian, Tingyang Xu, Weiyang Xie, Ying Wei, Wenbing Huang, and Junzhou Huang. 2020. Self-supervised graph transformer on large-scale molecular data. *Advances in Neural Information Processing Systems* 33 (2020), 12559–12571.

[22] Aravind Sankar, Xinyang Zhang, and Kevin Chen-Chuan Chang. 2017. Motif-based convolutional neural network on graphs. *arXiv preprint arXiv:1711.05697* (2017).

[23] Kristof T Schütt, Farhad Arbabzadah, Stefan Chmiela, Klaus R Müller, and Alexandre Tkatchenko. 2017. Quantum-chemical insights from deep tensor neural networks. *Nature communications* 8, 1 (2017), 13890.

[24] Teague Sterling and John J Irwin. 2015. ZINC 15–ligand discovery for everyone. *Journal of chemical information and modeling* 55, 11 (2015), 2324–2337.

[25] Arjun Subramonian. 2021. Motif-driven contrastive learning of graph representations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 15980–15981.

[26] Fan-Yun Sun, Jordan Hoffmann, Vikas Verma, and Jian Tang. 2019. InfoGraph: Unsupervised and Semi-supervised Graph-Level Representation Learning via Mutual Information Maximization. *ArXiv* abs/1908.01000 (2019).

[27] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy gradient methods for reinforcement learning with function approximation. *Advances in neural information processing systems* 12 (1999).

[28] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. 2018. Graph attention networks. In *International Conference on Learning Representations (ICLR)*.

[29] Petar Veličković, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2018. Deep graph infomax. *arXiv preprint arXiv:1809.10341* (2018).

[30] Daixin Wang, Zhiqiang Zhang, Yeyu Zhao, Kai Huang, Yulin Kang, and Jun Zhou. 2023. Financial Default Prediction via Motif-preserving Graph Neural Network with Curriculum Learning. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2233–2242.

[31] Junmei Wang and Tingjun Hou. 2011. Application of molecular dynamics simulations in molecular property prediction II: diffusion coefficient. *Journal of computational chemistry* 32, 16 (2011), 3505–3519.

[32] Sebastian Wernicke. 2006. Efficient detection of network motifs. *IEEE/ACM transactions on computational biology and bioinformatics* 3, 4 (2006), 347–359.

[33] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, and Chengqi Zhang. 2020. Self-Supervised Learning of Graph Neural Networks: A Unified Review. *ArXiv* abs/2011.15069 (2020).

[34] Zhenqin Wu, Bharath Ramsundar, Evan N Feinberg, Joseph Gomes, Caleb Geniesse, Aneesh S Pappu, Karl Leswing, and Vijay Pande. 2018. MoleculeNet: a benchmark for molecular machine learning. *Chemical science* 9, 2 (2018), 513–530.

[35] Jiaxuan Xu, Xin Wang, Ziwei Zhang, Yijian Qin, and Wenwu Zhu. 2020. Do not train it: A linear neural architecture search of graph neural networks. In *International Conference on Data Mining (ICDM)*. 1–10.

[36] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2019. How powerful are graph neural networks?. In *International Conference on Learning Representations (ICLR)*.

[37] Kevin Yang, Kyle Swanson, Wengong Jin, Connor Coley, Philipp Eiden, Hua Gao, Angel Guzman-Perez, Timothy Hopper, Brian Kelley, Miriam Mathea, et al. 2019. Analyzing learned molecular representations for property prediction. *Journal of chemical information and modeling* 59, 8 (2019), 3370–3388.

[38] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.

[39] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph contrastive learning with augmentations. *Advances in neural information processing systems* 33 (2020), 5812–5823.

[40] Matthew D Zeiler and Rob Fergus. 2014. Visualizing and understanding convolutional networks. In *Computer Vision–ECCV 2014: 13th European Conference, Zurich, Switzerland, September 6-12, 2014, Proceedings, Part I 13*. Springer, 818–833.

[41] Muhan Zhang and Yixin Chen. 2018. Link prediction based on graph neural networks. *Advances in neural information processing systems* 31 (2018).

[42] Wentao Zhang, Zeang Sheng, Ziqi Yin, Yuezihan Jiang, Yikuan Xia, Jun Gao, Zhi Yang, and Bin Cui. 2022. Model degradation hinders deep graph neural networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2493–2503.

[43] Zaixi Zhang, Qi Liu, Hao Wang, Chengqiang Lu, and Chee-Kong Lee. 2021. Motif-based graph self-supervised learning for molecular property prediction. *Advances in Neural Information Processing Systems* 34 (2021), 15870–15882.

[44] Ziwei Zhang, Yijian Qin, Xin Wang, and Wenwu Zhu. 2020. Designing the topology of graph neural networks: A novel feature fusion perspective. In *International Conference on Machine Learning (ICML)*. 11348–11358.

[45] Bolei Zhou, David Bau, Aude Oliva, and Antonio Torralba. 2018. Interpreting deep visual representations via network dissection. *IEEE transactions on pattern analysis and machine intelligence* 41, 9 (2018), 2131–2145.

[46] Jie Zhou, Zhen Zhang, Xin Wang, and Wenwu Zhu. 2020. Efficient and explainable graph neural architecture search via monte-carlo tree search. In *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML-PKDD)*. 1–17.

[47] Jinhua Zhu, Kehan Wu, Bohan Wang, Yingce Xia, Shufang Xie, Qi Meng, Lijun Wu, Tao Qin, Wengang Zhou, Houqiang Li, et al. 2022. $\mathcal{O}$-GNN: incorporating ring priors into molecular modeling. In *The Eleventh International Conference on Learning Representations*.

Unpublished working draft. Not for distribution.